

2025-26

Chapitre 4 : Les Boucles

Programmation

UE-3 : Informatique

Licence 2^{ème} Année

Parcours Sciences pour l'ingénieur

CHAPUIS Yves André
UNIVERSITE DE STRASBOURG

Cours librement inspiré du « Cours de Python » de Patrick FUCHS et
Pierre POULAIN, Université de Paris, 2021

1. Introduction

1.1. C'est quoi une boucle ?

En programmation, on est souvent amené à **répéter** plusieurs fois une instruction. Incontournables à tout langage de programmation, les **boucles** vont nous aider à réaliser cette tâche de manière compacte et efficace.

Les *boucles* vont nous permettre d'exécuter plusieurs fois un bloc d'instructions, c'est-à-dire d'exécuter un code « en boucle » tant qu'une **condition** donnée est vérifiée.

1.2. Exemple de boucle

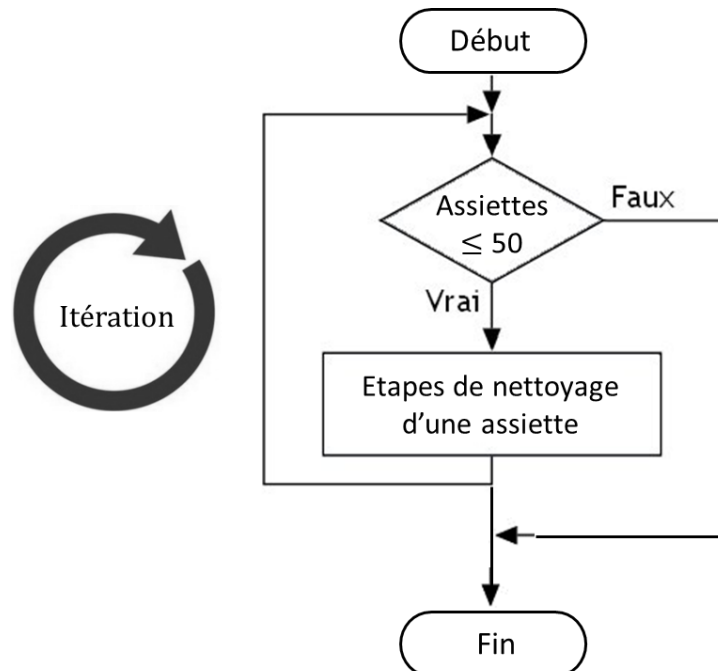
Un exemple simple de la vie quotidienne comme faire la vaisselle, dont voici les principales étapes :

1. Tu prends une assiette,
2. Tu la laves,
3. Tu la rinces,
4. Tu la poses à sécher,
5. Puis tu passes à l'assiette suivante.



Tu répètes **la même série d'actions pour chaque assiette**, jusqu'à ce qu'il n'en reste plus d'assiette à nettoyer.

Dans le cas concret de la vaisselle de **50 assiettes**, on peut donner le logigramme suivant :



On dit que la **vaisselle** correspond à une **boucle des** étapes de nettoyage des **50 assiettes**. Chaque boucle ou assiette correspond à une **itération**. Lorsque l'on atteint le nombre de 50 assiettes nettoyer, on **arrête** la vaisselle.

1.3. Programmation d'une boucle

Lorsqu'on programme, on va en effet souvent devoir exécuter plusieurs fois un même bloc d'instructions. Utiliser une *boucle* nous permet de n'écrire les instructions qu'on doit exécuter plusieurs fois qu'une seule fois.

Le fonctionnement général des *boucles* sera toujours le même et peut être partitionné en quatre parties, comme le décrit par le schéma de la figure 1 :

- **Début** : Début ou entrée de la *boucle* ;
- **Condition** : on pose une *condition* qui sera généralement liée à la valeur d'une variable;
- **Bloc d'instructions à répéter** : on exécute un *bloc d'instruction à répéter* de la *boucle* « en boucle » tant que la condition est vérifiée ;
- **Fin** : Fin ou sortie de la *boucle*.

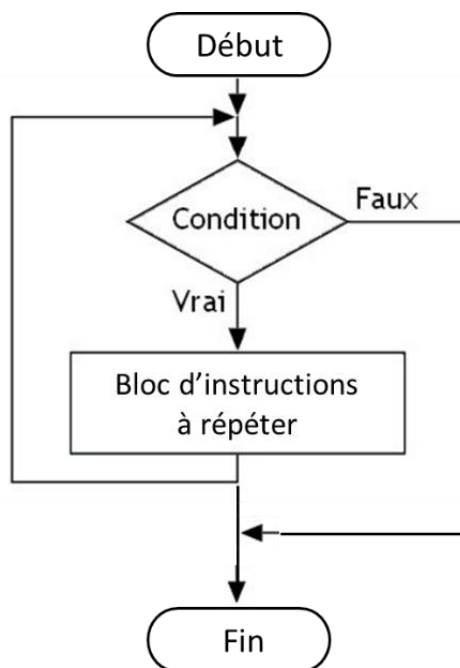


Figure 1. Description du fonctionnement général des boucles en schéma-bloc.

Pour éviter de rester bloqué à l'infini dans une boucle, vous pouvez donc déjà noter qu'il faudra que la condition donnée soit fausse à un moment donné (pour pouvoir sortir de la boucle).

Selon le type de condition, on va avoir différents moyens de faire cela. Nous allons voir les plus courants dans la suite de cette leçon.

Nous avons accès à deux boucles en Python :

- La boucle **for** → "pour...";
- La boucle **while** → "tant que..."

2. Boucle `for`

2.1. Principes

La **boucle** `for` énumère les éléments d'un ensemble et pour chaque **élément**, répète toujours le même code en fonction de cet élément.

Exemple :

Prenons une personne faisant ses courses dans un supermarché à partir d'une liste :



A chaque élément de liste, la personne devra faire en boucle les opérations suivantes :

1. Lire sur la liste l'objet à acheter,
2. Trouver l'objet dans le supermarché,
3. Mettre l'objet dans le chariot.
4. Aller payer le contenu du chariot, si la liste est complète.

Le code Python pour une telle boucle est :

```
1  for objet in liste_courses :
2      trouver(objet)
3      mettre_dans_le_chariot(objet)
4      aller_payer()
```

Pour chaque **objet** dans la **liste de courses**, on fait toujours la même chose :

- on trouve l'**objet** ;
- on le met dans le **chariot** ;
- on va **payer** quand il n'y a plus d'objet dans la liste de course.

Ligne 1 (en rouge) : désignation de la boucle « `for` »

La boucle est désignée par la ligne d'instruction débutant par «**for**», laquelle est terminée par le caractère **deux-points** « `:` ».

```
1  for objet in liste_courses :
2      trouver(objet)
3      mettre_dans_le_chariot(objet)
4      aller_payer()
```

Ligne 2 et 3 (en vert) : bloc d'instructions

La ligne précédente désignant la boucle `for` attend un **bloc d'instructions** signalé par l'**indentation** en l'occurrence toutes les instructions que Python répétera à chaque **itération** de la boucle. On appelle ce bloc d'instructions le **corps de la boucle**.

```
1  for objet in liste_courses :  
2      trouver(objet)  
3      mettre_dans_le_chariot(objet)  
4  aller_payer()
```

Ligne 4 (en bleu) : sortie de boucle

La fin de la boucle est signalée par l'instruction qui n'est pas indentée. Dans notre exemple, le « `aller_payer` » n'est **pas indenté**, ce qui signifie que la boucle n'itère plus et que l'on est **sortie de la boucle** :

```
1  for objet in liste_courses :  
2      trouver(objet)  
3      mettre_dans_le_chariot(objet)  
4  aller_payer()
```

A noter que si cette instruction était indentée, cela signifierait que pour chaque objet, on devrait payer avant de passer à l'objet suivant.

Remarque

Outre une meilleure lisibilité, les deux-points « : » et l'**indentation** sont formellement requis en Python. Même si on peut indenter comme on veut (plusieurs espaces ou plusieurs tabulations, mais pas une combinaison des deux), les développeurs recommandent l'utilisation de quatre espaces.

Faites-en sorte de configurer votre éditeur de texte favori de façon à écrire quatre espaces lorsque vous tapez sur la touche *tabulation*.

A retenir

En résumé, voici les notions à retenir pour l'utilisation des boucles dans la programmation Python :

- **Boucle** : Les boucles permettent de répéter des actions simplement et rapidement par itération.
 - **Itération** : Une itération est la répétition d'un bloc d'instructions.
 - **Bloc d'instructions** : Un bloc d'instructions est une instruction composée de plusieurs instructions qui se suivent.
 - **Variable d'itération** : La variable d'itération prend la valeur du nombre d'itération d'une boucle.
 - **Indentation** : Une indentation consiste en l'ajout de *tabulations* ou d'*espaces* pour faire ressortir un bloc d'instructions.
-

2.2. Fonction `range()`

Passons à des exemples plus informatiques. Nous allons nous intéresser principalement à l'**énumération de nombres** car c'est de loin la plus utile au début de la programmation des boucles.

Et pour créer une liste de nombres, nous allons principalement utiliser la **fonction `range()`**.

2.2.1. Définition

Par définition, la fonction `range()` crée une liste de n nombres allant de 0 à $(n - 1)$. A noter que l'on commence la liste par 0 et par conséquent l'on finit à $(n - 1)$ si l'on veut n nombres en tout. Par exemple :

```
1 for i in range(3):  
2     print("Bonjour !")  
  
Bonjour !  
Bonjour !  
Bonjour !
```

Commentons en détails ce qu'il s'est passé dans cet exemple :

- Une variable `i` est créée par Python la première fois que l'instruction contenant le **`for`** est exécutée (si elle existait déjà son contenu serait écrasé). Cette variable est appelée **variable d'itération**. On peut choisir le nom que l'on veut pour la variable.
- La variable d'itération `i` prend successivement les différentes valeurs de la liste créée par la fonction `range(3)` qui représente les chiffres 0, 1 et 2 correspondant aux 3 itérations de la boucle.
- À chaque itération de la boucle, le bloc d'instructions est exécuté, soit l'affichage de la chaîne de caractères "Bonjour !" suivi d'un retour à la ligne (par défaut).
- Une fois la dernière itération exécutée, la boucle se termine.

Remarque

- la variable d'itération `i` ne sera pas détruite et contiendra ainsi la dernière valeur de la liste de la fonction **`range(3)`** (ici le nombre 2).
 - Pour Python, la fonction `range()` est un **nouveau type**. Par exemple dans l'instruction **`x = range(3)`** la variable `x` est de type `range` (tout comme on avait les types **`int`**, **`float`** ou **`str`**) à utiliser spécialement avec les boucles.
-

2.2.2. Paramétrage

La fonction `range()` peut être paramétrée par deux nombres n_1 et n_2 , tel que `range(n1, n2)`, pour créer une liste de nombres allant de n_1 à $(n_2 - 1)$, comme par exemple :

– `range(1, 7)`
qui représente les chiffres 1, 2, 3, 4, 5 et 6.

La fonction `range()` peut également être paramétrée par deux nombres n_1 et n_2 et un pas tel que `range(n1, n2, pas)`, pour créer une liste de nombres allant de n_1 à $(n_2 - 1)$ en sautant de pas en pas, comme par exemple

– `range(1, 7, 2)`
qui représente les chiffres 1, 3 et 5.

2.3. Boucles `for` imbriquées

On peut bien sûr enchaîner les boucles.

Imaginons que nous voulions afficher tous les couples (x, y) possibles avec x et y des entiers entre 0 et 5. Une façon de faire serait de commencer par $x = 0$ et énumérer tous les couples $(0, y)$ pour y entre 0 et 5 puis faire de même pour $x = 1$ jusqu'à $x = 5$.

On voit bien qu'il faut faire en boucle sur x et une boucle sur y .

Et comme tout ce qui se trouve dans une boucle doit être indenté, il faudra donc décaler notre deuxième `for` par rapport au premier.

Voici un exemple de programme pour afficher ces couples :

```
1 for x in range(3):
2     for y in range(3):
3         print((x, y))
```

(0, 0)
(0, 1)
(0, 2)
(1, 0)
(1, 1)
(1, 2)
(2, 0)
(2, 1)
(2, 2)

On peut également faire dépendre la deuxième boucle de la variable de la première (puisque'elle est imbriquée).

Par exemple si on veut afficher les couples (x, y) avec $y \leq x$. On veut donc que y soit entre 0 et x ce qui donnerait comme code :

```
1 for x in range(3):
2     for y in range(x+1):
3         print((x, y))
```

(0, 0)
(1, 0)
(1, 1)
(2, 0)
(2, 1)
(2, 2)

2.4. Boucles `for` et chaînes de caractères

La boucle `for` va également nous permettre de créer une boucle en énumérant les lettres de texte (chaîne de caractères).

Par exemple si je veux afficher les chiffres d'un texte :

```
1 texte="Quatre bonbons à 0,25 euro chacun coûtent un euro"
2 for caractere in texte:
3     if caractere in "0123456789":
4         print(caractere)
```

0
2
5

Quelques explications :

La boucle `for` va prendre le premier caractère de texte et le placer dans la variable `caractère` c'est à dire que `caractère = 'Q'`. Ensuite on teste si `car` fait partie des chiffres. Comme ce n'est pas le cas, et qu'il n'y a rien à faire dans ce cas, on revient à la boucle `for` avec la deuxième lettre cette fois et on recommence jusqu'au caractère `"0"` qui fait partie des chiffres donc on l'affiche etc.

Nous reviendrons en détails dans la suite du cours sur l'utilisation des boucles pour le traitement des chaînes de caractères.

3. Boucle `while`

3.1. Principes

While signifie en anglais "tant que".

Tout comme la boucle `for`, la **boucle `while`** permet de répéter des instructions mais contrairement à **`for`** qui le fait en énumérant les éléments de quelque chose, **`while`** permet de boucler tant qu'une condition est vérifiée.

L'avantage de **`while`** est donc qu'il ne faut pas connaître à l'avance le nombre de fois où on devra répéter notre boucle.

Par exemple, observez le résultat pour le code suivant :

```

1  i = 0
2  while i < 4:
3      print(i)
4      i+=1

```

0
1
2
3

Remarquez qu'il est encore une fois nécessaire d'**indenter** le **bloc d'instructions** correspondant au corps de la boucle (ici, les instructions lignes 3 et 4).

Notez qu'une boucle `while` nécessite généralement **trois éléments** pour fonctionner correctement :

1. Initialisation de la **variable d'itération i** avant la boucle (ligne 1).
2. Test de la **variable d'itération i** associée à l'instruction `while` (ligne 2).
3. Mise à jour de la **variable d'itération i** dans le corps de la boucle (ligne 4).

3.2. Problème des « boucles infinies »

Faites bien attention aux tests et à l'incrémentation que vous utilisez car une erreur mène souvent à des « boucles infinies » qui ne s'arrêtent jamais. Par exemple :

```

1  i = 0
2  while i < 10:
3      print("Python c'est cool")

```

Python c'est cool
Python c'est cool
Python c'est cool
Python c'est cool
Python c'est cool
Python c'est cool
Python c'est cool

Ici, nous avons omis de mettre à jour la variable **i** dans le corps de la boucle. Par conséquent, la boucle ne s'arrêtera jamais puisque la condition `i < 10` sera toujours vraie.

Pratique

Pour stopper la boucle lorsque l'on utilise **Jupyter Notebook**, sélectionnez Kernel, puis Restart & Clear Output, puis Relancer et effacer toutes les sorties, comme le montre l'extrait ci-dessous.



4. Instructions `break`/`continue`/`pass`

En utilisant les boucles `for` et `while` en Python, vous pouvez automatiser et répéter efficacement des tâches.

Cependant, il arrive parfois qu'un facteur externe vienne influencer la façon dont votre programme fonctionne. Le cas échéant, vous voudrez éventuellement que votre programme quitte complètement une boucle, saute une partie d'une boucle avant de continuer, ou ignorer ce facteur externe.

Pour cela, vous pouvez utiliser les instructions :

- `break`
- `continue`
- `pass`

Les instructions `break`, `continue` et `pass` sont donc des outils supplémentaires de contrôle de flux dans les boucles.

4.1. Instruction `break`

Sous Python, l'instruction `break` vous donne la possibilité de **quitter une boucle** au moment où une condition externe est déclenchée.

Pour illustrer cette notion, nous programmerons l'instruction `break` dans le cas de boucles `for` et d'une boucle `while`, comme le montre la figure ci-dessous.

Dans les deux cas, l'instruction `break` se trouve dans le bloc d'instructions de la boucle, généralement après une instruction conditionnelle `if`.

Dès que le code rencontre l'instruction `break`, on quitte la boucle pour poursuivre un autre code.

Pour une boucle `for` :

```
for val in sequence:
    # code
    if condition:
        break
    # code
```



Pour une boucle `while` :

```
while condition:
    # code
    if condition:
        break
    # code
```



Examinons un exemple qui utilise l'instruction **break** dans une boucle **for** :

```
1 number = 0
2
3 for number in range(10):
4     if number == 5:
5         break      # break here
6
7     print('Number is ' + str(number))
8
9 print('Out of loop')
```

```
Number is 0
Number is 1
Number is 2
Number is 3
Number is 4
Out of loop
```

- **Ligne 1**

Dans ce court programme, le `number` de variables est initialisé à 0 (pas vraiment utile ici...)

- **Ligne 3**

Ensuite, une instruction **for** crée la boucle à condition que le **number** de variables soit inférieur à 10.

- **Ligne 4**

Dans la boucle **for** se trouve une instruction **if** qui présente la condition suivante : si le **number** de variables est équivalent à l'entier 5, alors on va à la ligne 4.

- **Ligne 5**

On lit l'instruction **break** et l'on sort de la boucle. La prochaine instruction lue par le programme sera celle de la **ligne 9**.

- **Ligne 7**

Dans la boucle se trouve également une instruction `print('Number is ' + str(number))` qui s'exécutera avec chaque itération de la boucle **for** jusqu'à ce que la boucle se brise, étant donné qu'elle se trouve après l'instruction **break**.

- **Ligne 9**

Pour savoir de quelle manière nous sommes en-dehors de la boucle, nous avons inclus une dernière instruction `print('Out of loop')` en dehors de la boucle **for**.

Cela montre qu'une fois le nombre entier évalué comme équivalent à 5, **la boucle se brise**, car le programme est instruit de le faire avec l'instruction **break**.

L'instruction **break** provoque la rupture d'une boucle d'un programme.

4.2. Instruction continue

Sous Python, l'instruction **continue** vous donne la possibilité de **passer la partie d'une boucle où une condition externe est déclenchée** mais tout de même passer à l'exécution du reste de la boucle.

C'est-à-dire que l'itération actuelle de la boucle sera interrompue, mais le programme reviendra au haut de la boucle.

Pour illustrer cette notion, nous programmerons l'instruction **continue** dans le cas de boucles **for** et d'une boucle **while**, comme le montre la figure ci-dessous. Dans les deux cas, l'instruction **continue** se trouve dans le bloc d'instructions de la boucle, généralement après une instruction conditionnelle **if**.

Dès que le code rencontre l'instruction **continue**, on quitte la boucle pour poursuivre un autre code.

Pour une **boucle for** :



```
for val in sequence:
    # code
    if condition:
        continue

    # code
```

Pour une **boucle while** :



```
while condition:
    # code
    if condition:
        continue

    # code
```

Examinons un exemple qui utilise l'instruction `break` dans une boucle `for` :

```
1 number = 0
2
3 for number in range(10):
4     if number == 5:
5         continue    # continue here
6
7     print('Number is ' + str(number))
8
9 print('Out of loop')
```

```
Number is 0
Number is 1
Number is 2
Number is 3
Number is 4
Number is 6
Number is 7
Number is 8
Number is 9
Out of loop
```

- On reprend le même programme que pour l'exemple de l'instruction `break` en remplaçant `break` par `continue`.
- Ici, `Number is 5` n'apparaît jamais dans la sortie, mais la boucle `continue` après pour imprimer des lignes des numéros 6 à 10 avant de quitter la boucle.
- On peut utiliser l'instruction `continue` pour éviter un code conditionnel profondément imbriqué ou optimiser une boucle en supprimant les cas fréquemment répétés que l'on souhaite rejeter.

L'instruction **`continue`** pousse un programme à sauter certains facteurs qui se trouvent dans une boucle, puis à continuer le reste de la boucle.

4.3. Instruction **pass**

Lorsqu'une condition externe est déclenchée, l'instruction **pass** vous permet de gérer la condition sans que la boucle soit affectée de quelque manière que ce soit ; tout le code continuera à être lu à moins qu'un **break** ou une autre instruction ne se produise.

Comme pour les autres instructions, l'instruction **pass** sera dans le bloc de code sous l'instruction de boucle, généralement après une instruction conditionnelle **if**.

En utilisant toujours le même bloc de code, remplaçons l'instruction **break** ou **continue** par une instruction **pass** :

```
1  nombre = 0
2  for nombre in range(10):
3      if nombre == 5:
4          pass # pass ici
5      print(f'Le Nombre est {nombre}')
6  print('En dehors de la boucle')
```

L'instruction **pass** qui se produit après l'instruction **if** conditionnelle indique au programme de continuer à exécuter la boucle et d'ignorer le fait que la variable `nombre` est réévaluée comme équivalente à 5 au cours de l'une de ses itérations. Nous allons exécuter le programme et considérer le résultat :

```
Le Nombre est 0
Le Nombre est 1
Le Nombre est 2
Le Nombre est 3
Le Nombre est 4
Le Nombre est 5
Le Nombre est 6
Le Nombre est 7
Le Nombre est 8
Le Nombre est 9
En dehors de la boucle
```

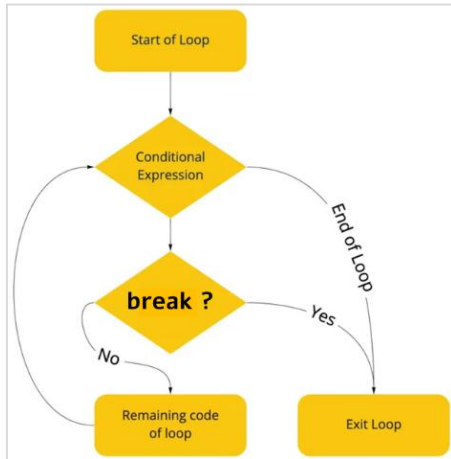
*En utilisant l'instruction **pass** dans ce programme, nous remarquons que le programme s'exécute exactement comme il le ferait s'il n'y avait pas d'instruction conditionnelle dans le programme.*

*L'instruction **pass** indique au programme de ne pas tenir compte de cette condition et de continuer à exécuter le programme comme d'habitude.*

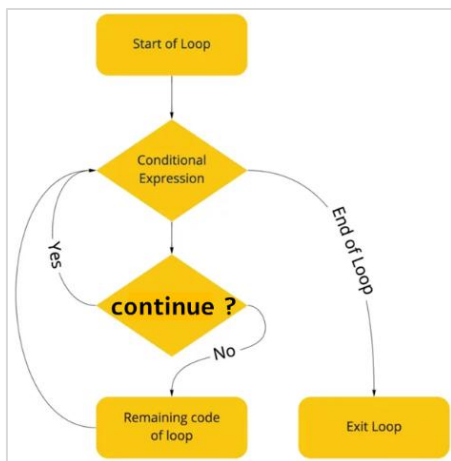
4.4. Synthèse

Les logigrammes ci-dessous reprennent les 3 instructions de contrôle de flux dans les boucles : **break**, **continue** et **pass**.

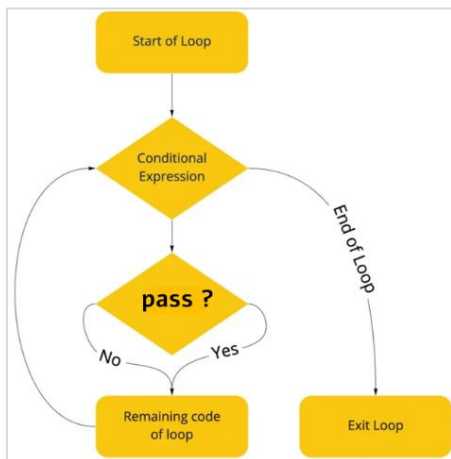
Break :



continue :



Pass :



5. Exercices

5.1. Affichage une séquence de nombres

Écrire un programme qui affiche les nombres de 1 à 10 à l'aide d'une boucle **for**.

5.2. Somme des nombres de 1 à n

À partir d'un nombre **n**, calculer la somme des nombres de 1 à **n** à l'aide d'une boucle **while**.

5.3. Afficher les nombres pairs jusqu'à 20

Afficher tous les nombres pairs entre 1 et 20 inclus.

5.4. Factorielle d'un nombre

Écrire un programme qui calcule la factorielle d'un nombre **n**.

5.5. Nombre premier

Vérifier si un nombre donné est premier.

Correction

5.1. Affichage une séquence de nombres

```
for i in range(1, 11):  
    print(i)
```

Output :

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

Explication :

- `range(1, 11)` génère les nombres de 1 à 10 (le 11 n'est pas inclus).
- La variable `i` prend chaque valeur, et `print(i)` les affiche.

5.2. Somme des nombres de 1 à n

```
1 n = 10  
2 somme = 0  
3 i = 1  
4  
5 while i <= n:  
6     somme += i  
7     i += 1  
8  
9 print(f"La somme de 1 à n est : {somme}")
```

Output :

```
La somme de 1 à n est : 55
```

Explication :

- La boucle `while` s'exécute tant que `i ≤ n`.
- À chaque itération, on ajoute `i` à la somme, puis on incrémente `i`.

5.3. Afficher les nombres pairs jusqu'à 20

```
1 for i in range(2, 21, 2):  
2     print(i)
```

Output :

```
2  
4  
6  
8  
10  
12  
14  
16  
18  
20
```

Explication :

- Le troisième paramètre de `range ( 2 )` est le **pas**.
- `range(2, 21, 2) → 2, 4, 6, ..., 20`.

5.4. Factorielle d'un nombre

```
1 n = 4  
2 fact = 1  
3  
4 for i in range(1, n + 1):  
5     fact *= i  
6  
7 print(f"La factorielle de n est : {fact}")
```

Output :

```
La factorielle de n est : 24
```

Explication :

- La factorielle est le produit des nombres de 1 à `n`.
- On initialise `fact` à 1 et on multiplie successivement.

5.5. Nombre premier

```
1 n = 10    # Choix du nombre à tester
2
3 if n < 2:
4     print("Ce n'est pas un nombre premier.")
5 else:
6     est_premier = True
7     for i in range(2, int(n**0.5) + 1):
8         if n % i == 0:
9             est_premier = False
10            break
11
12 if est_premier:
13     print(f"n = {n} est un nombre premier.")
14 else:
15     print(f"n = {n} n'est pas un nombre premier.")
```

Output :

n = 10 n'est pas un nombre premier.

Rappel :

Un **nombre premier** est un entier supérieur à 1 qui n'a que deux diviseurs :

👉 1 et lui-même.

Exemples :

- 2, 3, 5, 7, 11, 13 sont premiers.
- 4, 6, 8, 9, 10 ne le sont pas (car ils ont d'autres diviseurs).

Explication du programme :

1 | n = 10

- On choisit un nombre n

2 | if n < 2:

- Les nombres inférieurs à 2 (c'est-à-dire 0 et 1) **ne sont pas premiers**.
- Donc si c'est le cas, on affiche directement :

```
"Ce n'est pas un nombre premier."
```

3 | else:

- Si n est supérieur ou égal à 2, alors on doit **vérifier s'il a un diviseur autre que 1 et lui-même**.
-

4 `est_premier = True`

- On part du principe que le nombre est premier.
- On changera cette variable si on trouve un diviseur.

5 `for i in range(2, int(n**0.5) + 1):`

- On va tester tous les diviseurs possibles de 2 jusqu'à la racine carrée de `n`.

🧠 Pourquoi jusqu'à la racine carrée ?

- Si `n` a un diviseur supérieur à sa racine carrée, alors le diviseur complémentaire serait plus petit, et on l'aurait déjà trouvé.

👉 Cela rend le programme **plus rapide** (pas besoin d'aller jusqu'à `n-1`).

Exemple :

- Pour 49, $\sqrt{49} = 7$.
Si on ne trouve aucun diviseur ≤ 7 , alors 49 est premier.

6 `if n % i == 0:`

- On vérifie si `n` est divisible par `i`.
- Si le reste (`%`) est 0, cela veut dire que `i` divise `n`.

📖 Exemple :

- Si `n = 15` et `i = 3`, alors `15 % 3 = 0`
→ donc 15 n'est pas premier.

7 `est_premier = False`

- Si on trouve un diviseur, on met `est_premier` à `False`.

8 `break`

- On arrête immédiatement la boucle, car on a déjà prouvé que `n` n'est pas premier.
(Pas besoin de tester les autres nombres.)

9 Après la boucle :

```
if est_premier:
    print(f"n = {n} est un nombre premier.")
else:
    print(f"n = {n} n'est pas un nombre premier.")
```

- Si `est_premier` est resté `True` → aucun diviseur trouvé → le nombre est premier.
- Sinon → on a trouvé un diviseur → ce n'est pas un nombre premier.

Exemple : `n = 10` n'est pas un nombre premier